

Deep Neural Networks

Contents

Fundamentals of Neural Network	3
1. Why AI, machine learning, and deep learning exist	3
2. Learning paradigms	3
3. Data, labels, features, and ground truth	4
4. From biological inspiration to artificial neurons	5
5. Perceptrons and their computation	5
6. Linear separability and logic gates	6
7. From perceptrons to MLPs and DNNs	8
8. Weights, bias, activation, prediction, and loss	9
Why nonlinear activation matters	10
Choosing an activation and a model	10
Vanishing and exploding gradients	11
Comparing two candidate architectures	11
What the 12 September lecture added: output probabilities and loss	11
Worked forward-pass example	12
9. How a perceptron learns	12
Training and inference are different phases	13
10. Tools introduced in the lectures	13
11. Real-world applications	14
12. Common mistakes and quick recall	14
13. References and further reading	15
Supporting Sources	15
Further Reading	15

Fundamentals of Neural Network

1. Why AI, machine learning, and deep learning exist

Many useful problems are difficult to solve with a fixed list of hand-written rules. A navigation system must respond to changing traffic. An image-recognition system must recognize an object even when its size, angle, lighting, or background changes. Instead of describing every possible case in advance, we can build systems that learn useful patterns from examples.

Artificial intelligence (AI) is the broad field of building systems that perform tasks associated with intelligent behaviour. Machine learning (ML) is one approach within AI: a model learns patterns from data and uses those patterns to make predictions or decisions. Not every AI system must use machine learning.

Deep learning is an approach within machine learning that uses neural networks with multiple computational layers. These layers can learn increasingly useful representations from data. Deep learning is especially helpful when inputs such as images, text, audio, and video are difficult to describe with a small set of manually written rules.

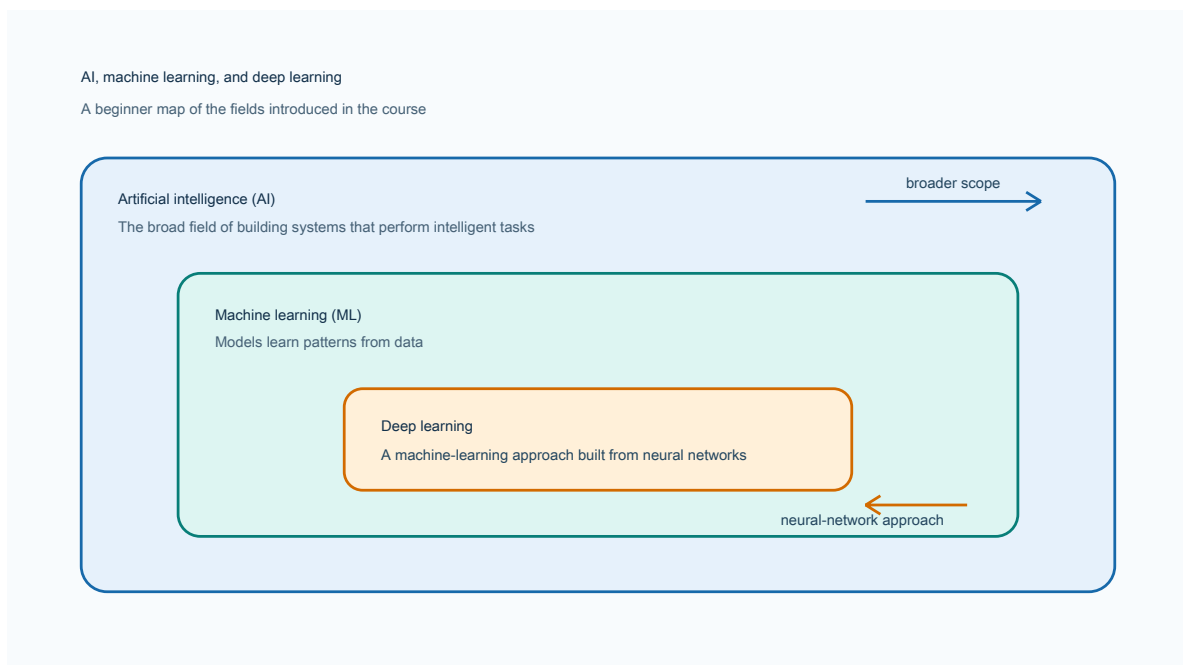


Figure 1. AI is the broad field; machine learning is one approach within AI; deep learning is a machine-learning approach built from neural networks.

2. Learning paradigms

A learning paradigm describes the kind of information available while a system learns. The lectures introduced four paradigms.

Paradigm	Information used for learning	Beginner example
Supervised learning	Examples contain input features and a known label or target	Learn to classify an image as a cat or dog from labeled images
Unsupervised learning	Examples have no supplied labels	Find groups of similar customers in purchasing data

Paradigm	Information used for learning	Beginner example
Semi-supervised learning	A smaller labeled set is combined with a larger unlabeled set	Learn from a few labeled images and many unlabeled images
Reinforcement learning	An agent acts in an environment and receives reward feedback	Learn actions in a simulated navigation task

The first three descriptions are mainly about whether examples have labels. Reinforcement learning uses a different framing: an agent interacts with an environment, chooses actions, and receives rewards or penalties. A reward is feedback about the result of an interaction; it is not simply a target label attached to every example.



Figure 2. Supervised, unsupervised, and semi-supervised learning differ in label availability; reinforcement learning is organized around interaction and reward feedback.

3. Data, labels, features, and ground truth

A model learns from a dataset, which is an organized collection of examples. One example might be a row describing a house, an image of a road, or a piece of text.

- A feature is an input value that may help the model. For a house, features could include floor area, number of rooms, and location.
- A label or target is the expected output in supervised learning. For house-price prediction, the target is the known sale price. For image classification, it may be the class name.
- Ground truth is the trusted correct answer used to compare with a model's prediction. In a labeled training example, the label normally serves as the ground truth.
- Annotation is the process of adding labels to data. An image can be annotated with a class name or with a box marking where an object appears.

Data can also be described by its form. Structured data fits an organized layout such as a table with named columns. Unstructured data includes forms such as images, free text, audio, and video that do

not naturally arrive as simple table rows. Structure and labeling are separate properties: an image can be labeled or unlabeled, and a table can also be labeled or unlabeled.

In supervised learning, each training example pairs features with a target. During prediction, the model receives new features and estimates the target without being shown the answer first.

4. From biological inspiration to artificial neurons

Neural networks take limited inspiration from biological nervous systems. In a simplified biological description, dendrites receive signals, the axon hillock at the junction of the cell body and axon integrates incoming activity, and the axon carries the resulting signal toward other cells.

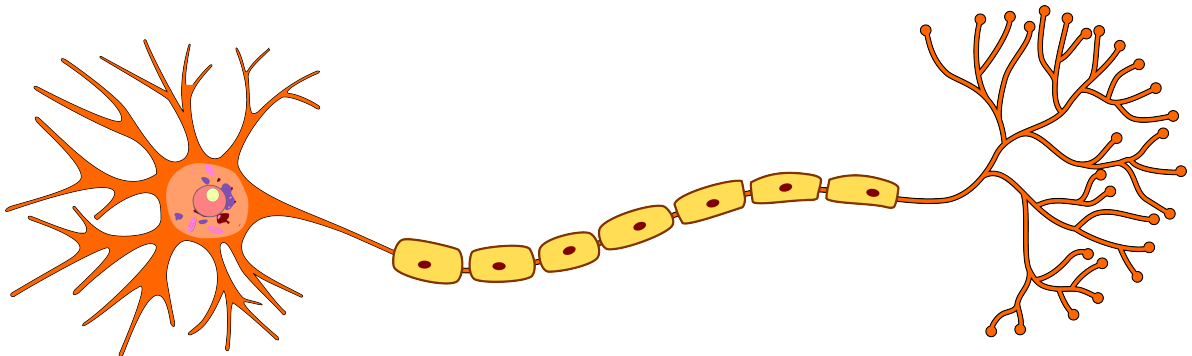


Figure 3. Structure of a biological neuron. Image by [Sanu N on Wikimedia Commons](#), used unmodified under [CC BY-SA 4.0](#).

An artificial neuron is a small mathematical unit, not a biological copy. It receives numerical inputs, gives each input a numerical weight, adds the results with a bias, and applies a function to produce an output. A network is formed by connecting many such units.

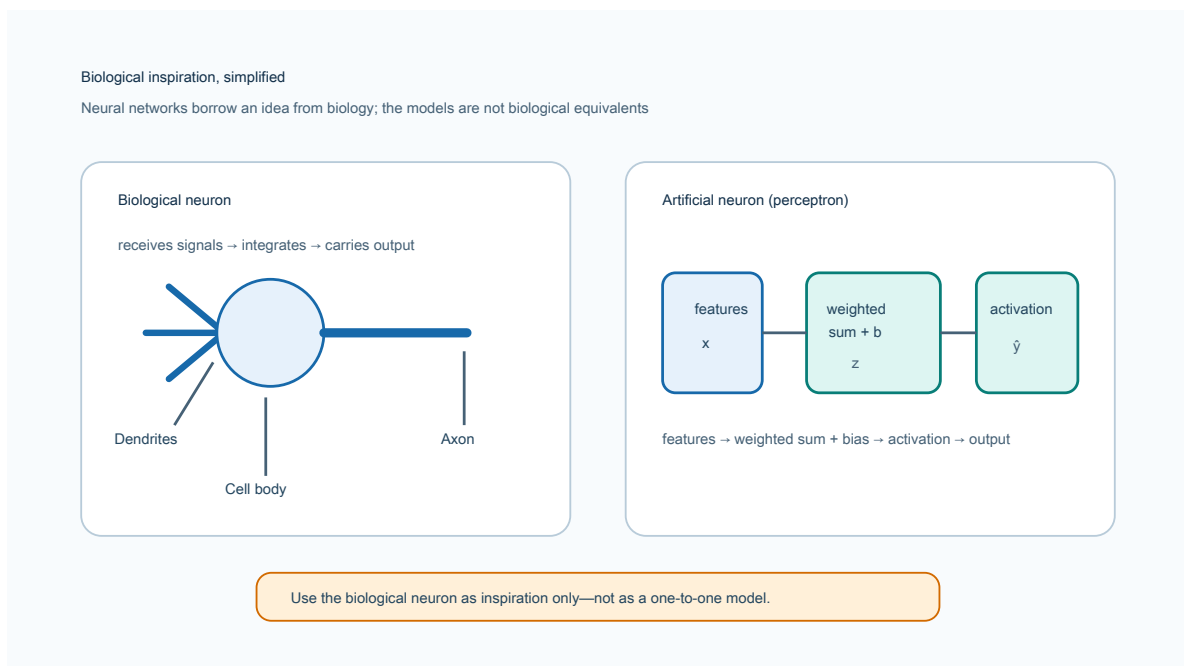


Figure 4. The biological neuron motivates the receive–combine–output idea, while the artificial neuron is a deliberately simplified mathematical model.

5. Perceptrons and their computation

A perceptron is a simple artificial neuron used to make a prediction from input features. It first combines the features and then applies an activation rule.

Suppose one example has several features. The symbol x_i means feature number i , and w_i means the weight paired with that feature. The index i identifies a position in the feature list. The symbol b is the bias, z is the combined value before activation, and Σ means to add the feature-weight products for all feature positions. The function f is the activation function, and $\hat{y}$ (read “y-hat”) is the prediction.

The weighted calculation is:

$$z = \sum_i x_i w_i + b$$

The prediction is:

$$\hat{y} = f(z)$$

For binary output, a simple step activation can use this rule:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

For example, take two features with values $x_1 = 1$ and $x_2 = 0$, weights $w_1 = 0.7$ and $w_2 = 0.4$, and bias $b = -0.2$. The combined value is $z = (1 \times 0.7) + (0 \times 0.4) - 0.2 = 0.5$. Because z is at least zero, the step activation produces $\hat{y} = 1$.

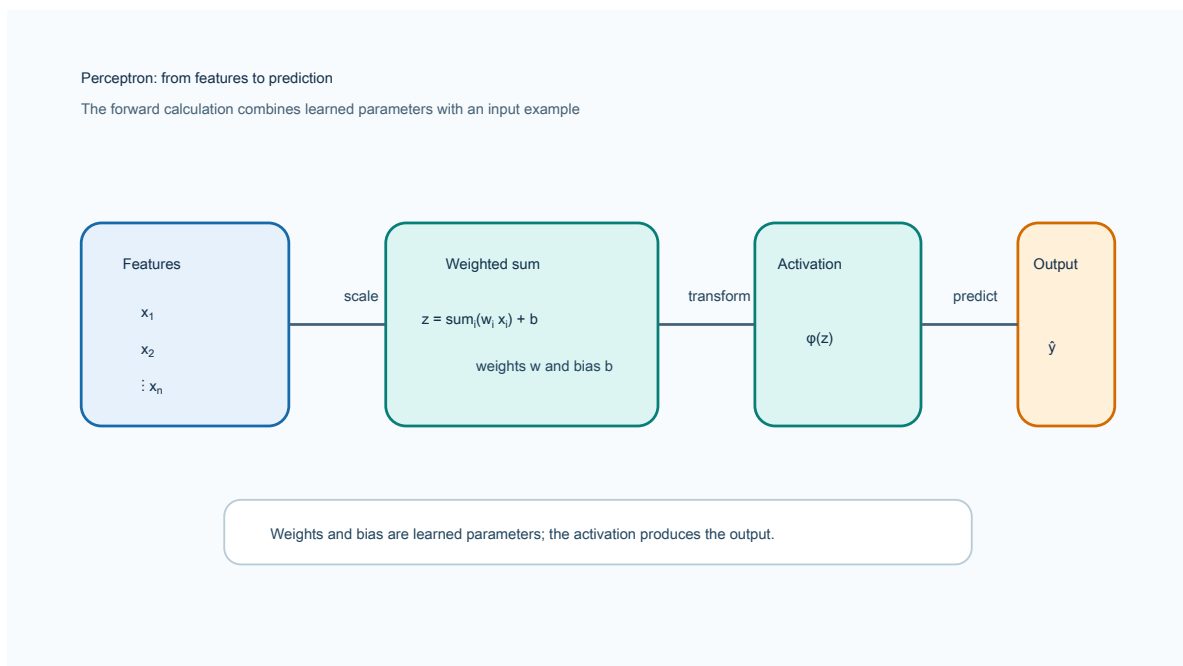


Figure 5. A perceptron scales features with weights, adds a bias, and passes the combined value through an activation function.

6. Linear separability and logic gates

A binary classifier places examples into one of two classes. Its decision boundary is the dividing line between the classes. With two features, a linear boundary is a straight line; with more features, the same idea extends to a flat boundary in a higher-dimensional space.

Data is linearly separable when one linear boundary can separate the classes. A single perceptron with a linear weighted calculation and threshold makes this kind of boundary.

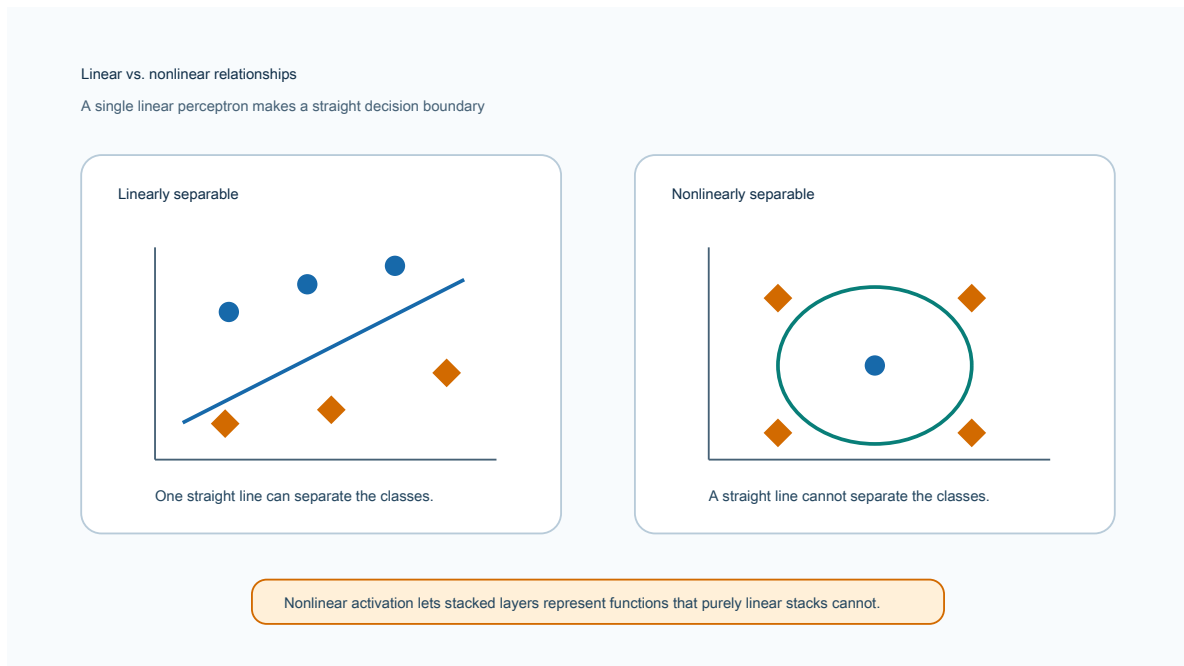


Figure 6. One straight boundary is enough for linearly separable data, but not for a nonlinear class arrangement.

The AND logic gate outputs 1 only when both binary inputs are 1.

x_1	x_2	AND output y
0	0	0
0	1	0
1	0	0
1	1	1

These four points are linearly separable. For example, the step perceptron from the previous section can model AND with $w_1 = 1$, $w_2 = 1$, and $b = -1.5$.

The XOR logic gate outputs 1 when the two inputs are different.

x_1	x_2	XOR output y
0	0	0
0	1	1
1	0	1
1	1	0

The XOR classes occupy opposite corners. No single straight line separates the 1 outputs from the 0 outputs. Therefore, a single linear-threshold perceptron cannot model XOR. A multilayer network with nonlinear activation functions can combine multiple boundaries and represent XOR.



Figure 7. AND can be separated by one straight boundary; XOR cannot, which motivates multilayer nonlinear networks.

7. From perceptrons to MLPs and DNNs

One perceptron can represent only a limited set of relationships. A neural network connects many artificial neurons so that the output of one layer becomes input to another.

- The input layer receives the original features.
- A hidden layer transforms information between input and output. It is called hidden because its values are internal to the model rather than supplied as targets.
- The output layer produces the final prediction.

A multilayer perceptron (MLP) is a feed-forward network with an input layer, one or more hidden layers, and an output layer. In the course's beginner framing, a deep neural network (DNN) has more than one hidden layer. Here, “deep” refers to multiple computational layers; it does not mean that the model is biologically deeper or automatically better.

When reading an architecture diagram, count input features, hidden layers, units in each hidden layer, and output units separately. A common convention counts trainable transformation layers but not the input placeholder; always state the convention because some descriptions count layers differently. Width is the number of units in a layer, while depth is the number of successive computational transformations.

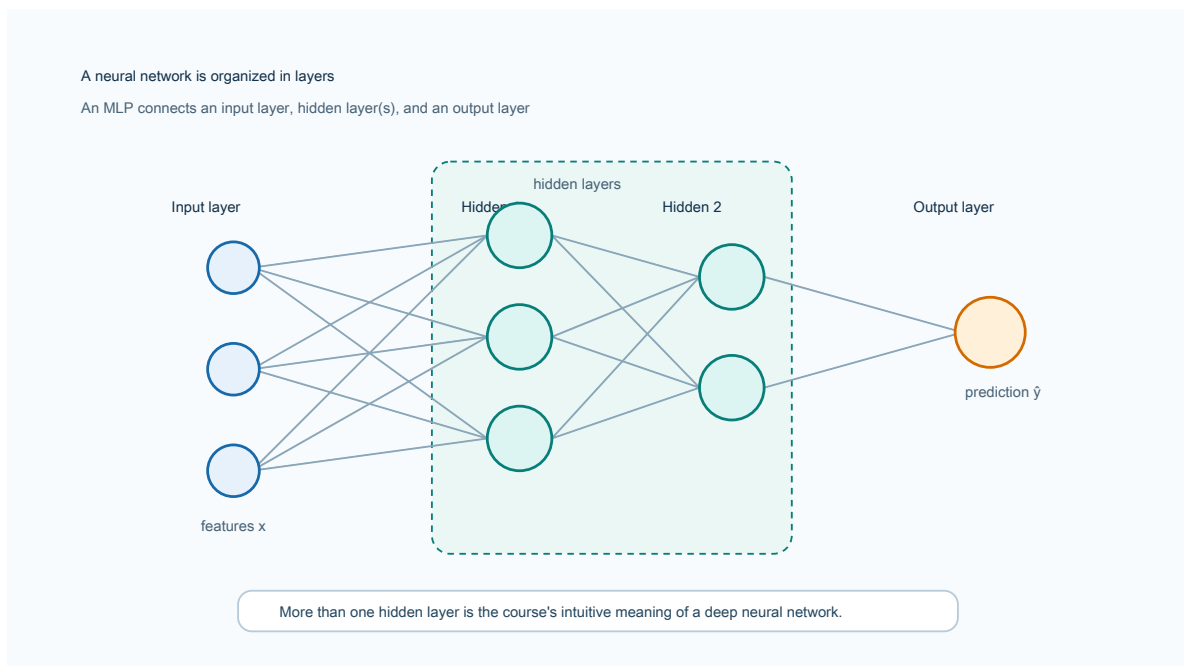


Figure 8. An MLP organizes connected units into input, hidden, and output layers; multiple hidden layers provide the course's introductory view of depth.

A universal-approximation result says that, under stated conditions and with enough hidden units, a feed-forward network with a suitable nonlinear activation can approximate continuous functions on compact domains arbitrarily closely. This is a representation result, not a promise that training will find the required parameters, that a small network will suffice, or that the model will generalize from finite data.

The lectures also named convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers as later architecture families. At this stage, those names are orientation only; their internal designs are outside the depth covered here.

8. Weights, bias, activation, prediction, and loss

Before a model can learn, its inputs must be represented as useful features. Feature engineering means selecting, creating, or transforming input values so they describe the problem more effectively. Traditional machine-learning workflows often rely heavily on manually designed features. Neural networks can learn useful intermediate representations, but they still need suitable input data.

A perceptron's weights control how strongly each feature influences the combined value. A larger positive weight pushes the result upward when its feature is present; a negative weight pushes it in the opposite direction. The bias shifts the activation threshold and lets the model make a suitable decision even when feature values are zero. Weights and biases are the model's learned parameters.

An activation function transforms the combined value. The simple step function makes a binary decision. Nonlinear activation functions are especially important in multilayer networks: without nonlinearity, stacking linear calculations still behaves like one linear calculation and cannot represent relationships such as XOR.

A prediction is the model's estimated output. During supervised training, the prediction is compared with the target. A loss function turns that difference into a number that measures prediction error; the symbol L is commonly used for this loss. A smaller loss means the prediction is closer according to the chosen loss function.

For a house whose known price is 600,000, a prediction of 500,000 has a noticeable gap. Loss represents such error in a form that an optimization method can use. The exact loss function depends on the task, so loss should not be read as one universal percentage or one universal calculation.

Why nonlinear activation matters

If every layer performs only a linear calculation, several layers collapse into one equivalent linear calculation. Depth alone therefore does not create a nonlinear decision boundary. A nonlinear activation between layers allows the network to bend and combine boundaries.

Activation	Rule or range	Beginner interpretation
Step	outputs 0 or 1 at a threshold	useful for the classical perceptron decision
Sigmoid	maps any score into (0, 1)	useful when an output is interpreted as a binary probability
Tanh	maps any score into (-1, 1)	produces centered positive and negative activations
ReLU	$\max(0, z)$	keeps positive values and sets negative values to zero
Softmax	maps a vector of scores to positive values that sum to 1	useful for a mutually exclusive multiclass output

The activation should match the layer's role. Hidden layers commonly use a nonlinear function such as ReLU. The output layer depends on the task: a regression output may remain linear, a binary classifier may use sigmoid, and a mutually exclusive multiclass classifier may use softmax. The values entering sigmoid or softmax are logits; after the transformation they can be interpreted as model probabilities under the chosen model and training setup.

Choosing an activation and a model

Choose the output activation from the meaning of the target, not from the fact that a function is popular.

- For a numerical regression target such as temperature or house price, a linear output is usually appropriate because the prediction is not restricted to a probability interval.
- For one binary decision, sigmoid gives one value between zero and one that can be interpreted as the positive-class probability. A step output is useful for a hard yes/no decision, but it discards confidence and is not differentiable at its threshold.
- For exactly one class among several mutually exclusive classes, softmax gives one probability per class and the probabilities add to one.
- For several labels that may be true at the same time, use one independent binary output per label rather than forcing the outputs to sum to one.
- Hidden layers need a nonlinear activation to represent nonlinear relationships. ReLU is a common choice; sigmoid and tanh can be useful when their bounded or centered ranges suit the design.

The sign activation records only whether a score is negative or positive. It is useful for explaining a threshold decision, but its hard threshold is not a convenient choice for ordinary gradient-based hidden-layer training. The step activation similarly returns one side of a threshold or the other. These

functions should not be confused with sigmoid, which retains a graded value between zero and one.

Vanishing and exploding gradients

During backpropagation, a gradient is passed through many layers. If repeated derivatives are mostly smaller than one, the product can become extremely small; this is a vanishing gradient. Early layers then receive little useful update. If repeated derivatives or weights are large, the product can become extremely large; this is an exploding gradient, which can make updates unstable or produce non-finite values.

Saturating activations such as sigmoid and tanh can contribute to vanishing gradients when their inputs are far from the central part of their curves. ReLU avoids saturation on its positive side but can leave a unit inactive for negative inputs. Initialization, normalization, activation choice, learning rate, and gradient clipping are separate tools for diagnosing or controlling these problems; none is a universal cure.

Comparing two candidate architectures

More neurons or more layers do not automatically make a model better. A larger network can represent more complicated relationships, but it also has more parameters, needs more computation and data, and can overfit. A smaller network may train more easily and generalize better when the task is simple, but it may underfit a genuinely nonlinear problem.

To justify a choice, compare the candidates on the same task and validation protocol. Check whether the output activation matches the target, whether the architecture can represent the decision boundary, whether training is stable, and whether validation performance—not training performance alone—supports the choice. State the trade-off: capacity, computation, data requirements, stability, and generalization.

What the 12 September lecture added: output probabilities and loss

The lecture revisited activation functions as the link between a neuron's combined value and the value passed onward. For a mutually exclusive multiclass prediction, softmax converts the output scores into non-negative class probabilities whose total is one. The largest probability identifies the model's chosen class, while the full probability vector shows uncertainty across all classes.

Stable softmax + cross-entropy

$$u(k) = a(k) - \max_j a(j) \quad L = -\log(p(y))$$
$$p(k) = \frac{e^{u(k)}}{\sum_j e^{u(j)}}$$

Each $p(k)$ lies between 0 and 1; all $p(k)$ values sum to 1.

The lecture also used log-likelihood loss, commonly called cross-entropy loss, for classification. It is computed after the output layer has produced its class probabilities; it is not an activation function and it is not a replacement for the forward pass. During training, the loss is measured on the training examples and monitored separately on validation examples. The optimizer then uses the loss gradients to update the weights and biases. Keep these stages separate: forward calculation → output activation → loss calculation → backward gradient information → parameter update.

Worked forward-pass example

Consider one hidden unit with inputs $x_1 = 2$ and $x_2 = 1$, weights $w_1 = 0.5$ and $w_2 = -1$, bias $b = 0.5$, and ReLU activation. First calculate the pre-activation value:

$$z = 0.5(2) + (-1)(1) + 0.5 = 0.5$$

Then apply the activation:

$$h = \max(0, z) = 0.5$$

The value h is not yet a class label or probability. It is an internal representation that can become an input to the next layer. This separation between pre-activation z and activation h is important when following a network calculation.

9. How a perceptron learns

Training repeats a simple idea: make a prediction, measure the error, and adjust parameters so later predictions can improve. One full pass through the training dataset is called an epoch.

For the classical binary perceptron update, let y be the correct label, $\hat{y}$ the predicted label, η (eta) a small positive learning rate that controls the update size, x_i feature number i , and w_i its weight. The arrow $\leftarrow$ means “replace the old value with the value on the right.” The update is:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

If the prediction is correct, $y - \hat{y}$ is zero and this rule does not change the weight. If the prediction is wrong, the feature value, error direction, and learning rate determine the adjustment. The bias can be adjusted in the same error-reducing spirit.

For a finite, linearly separable training set that is repeatedly presented under the standard perceptron procedure, the perceptron convergence theorem guarantees that only finitely many mistakes are made before a separating parameter choice is reached. The guarantee does not apply to nonseparable data such as XOR. In that case the updates may continue without finding parameters that classify every training example correctly.

For neural networks more generally, the calculation from features to prediction is the forward pass or forward propagation. The loss L is then measured. An optimizer updates weights and biases to reduce that loss. A gradient describes how a small parameter change affects loss. The notation $\partial L / \partial w$ means the loss gradient for a weight w , and $\partial L / \partial b$ means the loss gradient for the bias b . A conceptual gradient update is:

$$w \leftarrow w - \eta(\partial L / \partial w)$$

$$b \leftarrow b - \eta(\partial L / \partial b)$$

In a multilayer network, backpropagation passes loss-gradient information backward through the layers so the optimizer can update their parameters. This is the role of backpropagation at the depth introduced in the lectures; a calculus derivation is not needed here.

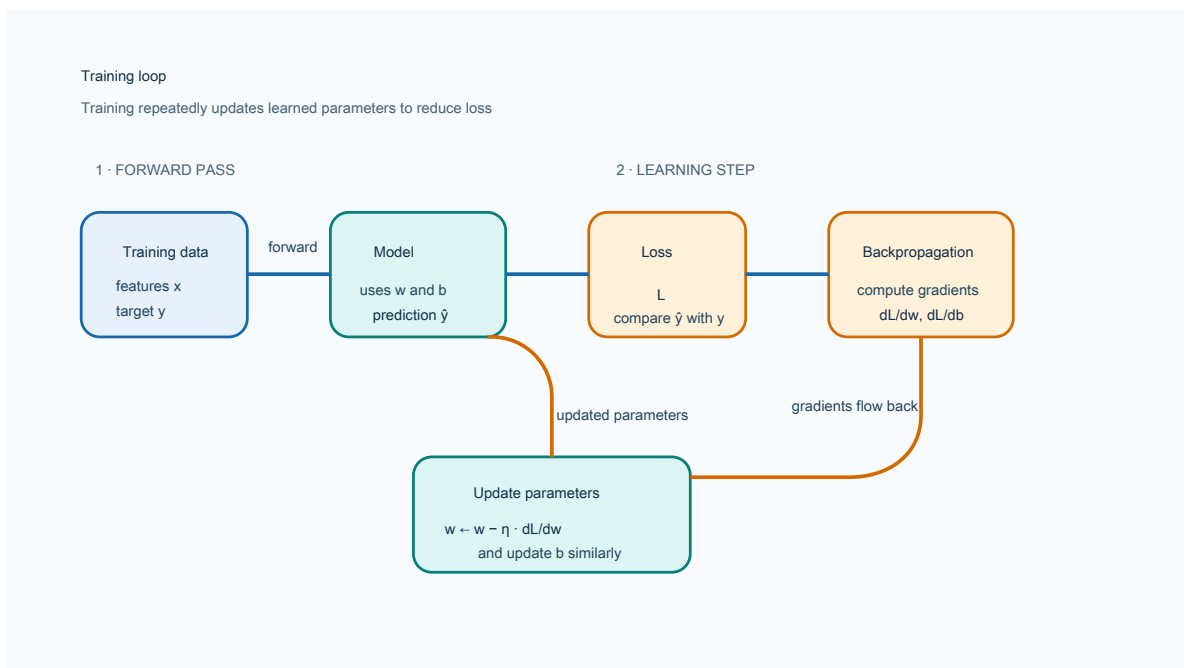


Figure 9. Training alternates a forward calculation with a learning step that uses error information to update parameters.

Training and inference are different phases

- During training, examples normally include targets. The network performs a forward pass, measures loss, computes gradients, and updates parameters.
- During inference, the learned parameters are fixed. The network receives new features and performs only the forward calculation needed to produce a prediction.

An epoch belongs to training, not inference. Likewise, backpropagation updates the model during training; it is not required merely to use the trained model for prediction.

10. Tools introduced in the lectures

The lectures introduced tools that support experimentation and implementation. They are parts of a workflow, not competing theories.

Tool	Introductory role
Google Colab	Run Python notebooks in a browser-hosted environment
Jupyter Notebook	Combine executable Python cells, results, text, and plots
NumPy	Work with numerical arrays and mathematical operations
Pandas	Load, organize, inspect, and transform tabular data
Matplotlib	Plot data, model behaviour, and results
scikit-learn	Use common machine-learning algorithms and utilities

Tool	Introductory role
TensorFlow	Build and train machine-learning and neural-network models
Keras	Use a high-level API for defining and training TensorFlow models
PyTorch	Build and train neural networks with tensor operations and layers

Colab or Jupyter provides the notebook environment. NumPy, Pandas, and Matplotlib support numerical work, data handling, and visualization. scikit-learn provides established machine-learning tools, while TensorFlow/Keras and PyTorch provide building blocks for neural networks. Framework choice does not change the underlying ideas of features, parameters, activations, prediction, loss, and optimization.

11. Real-world applications

Neural networks are useful when a task involves many interacting signals or complex data. The lectures used several motivating examples:

- House-price prediction: combine features such as area, rooms, and location to estimate a numerical price.
- Image recognition: learn visual patterns that help classify objects despite changes in viewpoint or appearance.
- Autonomous driving: combine observations from cameras, lidar, and radar to support perception of roads and nearby objects. This is a motivating application, not a claim that one neural network alone provides a complete or perfectly safe driving system.
- Event cameras: unlike conventional cameras that produce complete image frames at fixed intervals, event-based sensors asynchronously report brightness changes at individual pixels. The lectures introduced this only as an orientation to a different kind of vision data.
- Recommendation: use patterns in past interactions to suggest potentially relevant items.
- Translation: model patterns between languages to generate context-sensitive translations.
- Healthcare: use complex data such as medical images or measurements to support pattern recognition, while leaving clinical decisions and validation to the appropriate healthcare process.

The same foundation appears across these examples: choose useful inputs, process them through a model, produce a prediction, compare it with suitable feedback when training, and improve the model's parameters.

12. Common mistakes and quick recall

- AI, ML, and deep learning are not synonyms. Deep learning is one family inside machine learning, which is one approach inside AI.
- A weight is not a feature. A feature comes from the example; a weight is a learned model parameter.
- A hidden layer is not hidden data. It is an internal computational layer whose activations are produced by the model.

- More layers do not guarantee a better model. Architecture, data quality, optimization, regularization, and evaluation all matter.
- A prediction is not automatically a probability. Its meaning depends on the output activation and training objective.
- Low training loss is not enough. The model must also perform well on representative unseen data.

The central calculation is easy to remember: combine features with weights and bias, apply an activation, compare the prediction with the target during training, and update parameters using error information.

13. References and further reading

Supporting Sources

- [Google for Developers — What is Machine Learning?](#) supports the distinctions among supervised, unsupervised, and reinforcement learning.
- [OpenAI Spinning Up — Key Concepts in Reinforcement Learning](#) explains the agent, environment, action, and reward framing.
- [Dive into Deep Learning — Multilayer Perceptrons](#) supports hidden layers, nonlinear activation, and MLP structure.
- [scikit-learn — Neural network models \(supervised\)](#) describes weighted sums, biases, activations, loss, and gradient-based training.
- [OpenStax — Neurons and Glial Cells](#) supports the limited dendrite, axon-hillock, and axon analogy.
- [MIT OpenCourseWare — Linear Neural Networks](#) illustrates why one linear perceptron cannot solve XOR and how a layered network can.
- [Waymo — Driver Handbook: Perception](#) provides official context for cameras, lidar, radar, and sensor fusion in autonomous driving.
- [Gallego et al. — Event-based Vision: A Survey](#) validates the introductory event-camera terminology.

Further Reading

- [Google Machine Learning Crash Course](#) gives beginner explanations of datasets, loss, and optimization.
- [scikit-learn User Guide](#) introduces supervised and unsupervised learning tools and workflows.
- [TensorFlow — Keras Guide](#) introduces layers, models, training, losses, and optimizers through Keras.
- [PyTorch — Build the Neural Network](#) gives a beginner-oriented introduction to neural-network layers and models.

[Detailed](#) · [Know More](#) · [External Sources](#)